“Fake” is the best option here, at least as a default (the mode can be overridden in individual tests). This will require our tests to explicitly force jobs to execute, which can couple tests to the jobs being run, but it also means our tests reflect reality. In “Inline” mode, you can’t look at a test and understand how jobs interact with the feature. Inline mode also makes it difficult or impossible to make assertions about the state of the system after a job has been queued but before it has been executed. For a system test, this could be critical as a user may see a different UI while a job is executing.

To enable testing mode, you’ll need to require `sidekiq/testing` in `test/test_helper.rb`:

snapshots/1-3/sidekiq-book/test/test_helper.rb

```
  require_relative "../config/environment"
  require "rails/test_help"
  require "minitest/autorun"
» require "sidekiq/testing"
  require "minitest/mock"
  require "factory_bot"
```

Fake mode is the default when Sidekiq’s testing support is enabled, so you don’t need to do any further configuration to enable it. But you do want to make sure there are no jobs from a previous run still in the queue. To do that, call `Sidekiq::Worker.clear_all` inside `setup` in the base test case:

snapshots/1-4/sidekiq-book/test/test_helper.rb

```
  # Add more helper methods to be used by all tests here...
  setup do
    Rails.cache.clear
»   Sidekiq::Job.clear_all
  end
end
```

The test can now execute the jobs so the assertions can succeed. In Fake mode, Sidekiq adds the method `drain` to all job classes that will execute all queued jobs of that class. Sidekiq also provides `Sidekiq::Worker.drain_all`, which